FINTRUST

Transaction Insights and Risk Monitoring System

Pre-Build Project Definition and Development Plan

1. Project Overview

FinTrust is a cloud-native transaction insights and risk monitoring application. It receives transaction exports, validates and stores transaction data, calculates risk scores, exposes information through REST APIs, and notifies the compliance team when configured risk thresholds are breached.

Learners will not only design the solution; they will progressively develop, test, integrate, deploy, and monitor the application.

2. Business Problem

FinTrust operations teams receive transaction data that must be processed consistently and reviewed for potentially risky activity. Manual processing is slow, difficult to scale, and can make it harder to identify high-risk patterns quickly. FinTrust will automate the flow from transaction ingestion to risk scoring and alerting.

3. Project Objective

The application must be able to:

- Receive transaction data from CSV files.
- Validate transaction records before they are stored.
- Separate invalid records from valid records.
- Store customers, accounts, and transactions in a relational database.
- Apply rule-based transaction risk scoring.
- Store risk scores for fast retrieval.
- Expose transaction and risk information through REST APIs.
- Run scheduled risk-processing workflows.
- Send email alerts when risk thresholds are breached.
- Provide logging, monitoring, alarms, and request tracing.
- Support automated testing and controlled deployment.
- Package and deploy application components to AWS.

4. Project Scope

In Scope

- Transaction CSV ingestion and validation.
- Invalid-record quarantine handling.
- Customer, account, and transaction persistence.
- Rule-based risk scoring and classification.
- Risk-score storage and retrieval.
- REST API development.
- Scheduled batch processing.
- Risk alert email notifications.

- Cloud monitoring and tracing.
- Automated application testing.
- AWS deployment and technical documentation.

Out of Scope

- Real banking payment processing or movement of funds.
- Use of real customer financial data.
- Machine-learning fraud detection in the first release.
- A complete public banking portal.
- Automated real-world compliance decisions.

5. High-Level Application Flow

1. The operations team uploads a transaction CSV file.
2. Amazon S3 receives the file.
3. An S3 event invokes an AWS Lambda ingestion function.
4. The Lambda function validates the transaction records.
5. Invalid records are quarantined, and valid records continue for processing.
6. Valid transaction data is stored in Amazon RDS for PostgreSQL.
7. The risk engine evaluates the transactions against defined business rules.
8. Risk scores are stored in Amazon DynamoDB for fast lookup.
9. Amazon API Gateway exposes REST endpoints used by internal applications or dashboards.
10. Amazon EventBridge schedules nightly processing.
11. AWS Step Functions orchestrates multi-step batch processing.
12. Amazon SES sends alerts for transactions that breach the risk threshold.
13. Amazon CloudWatch and AWS X-Ray provide monitoring and tracing.

6. Technology Stack

Area	Technology / Service	Purpose
Programming	Python	Core application development
API	FastAPI	REST API development and local API testing
ORM	SQLAlchemy	Database models and data access
Relational data	PostgreSQL / Amazon RDS	Transaction ledger
Fast lookup	Amazon DynamoDB	Risk-score retrieval
File storage	Amazon S3	CSV landing and quarantine storage
Compute	AWS Lambda	Event-driven application functions
API exposure	Amazon API Gateway	Managed REST API entry point
Workflow	AWS Step Functions	Multi-step processing orchestration
Scheduling	Amazon EventBridge	Nightly and event-driven processing
Notifications	Amazon SES	Compliance email alerts

Monitoring	Amazon CloudWatch	Logs, metrics, dashboards, and alarms
Tracing	AWS X-Ray	Distributed request tracing
Configuration	SSM Parameter Store	Protected application configuration
Security	AWS IAM	Roles and least-privilege permissions
Testing	Pytest + API testing	Automated quality validation
Source control	Git + GitHub	Version control and collaboration

7. System Components

Component	AWS Services	Responsibility
Ingest Layer	S3 + Lambda	Receive and validate transaction CSV data.
Data Layer	RDS PostgreSQL + DynamoDB	Store the transaction ledger and risk scores.
Processing Layer	Lambda + Step Functions	Calculate scores and orchestrate batch processing.
API Layer	API Gateway + Lambda	Expose application functionality through REST APIs.
Alerts Layer	SES + EventBridge	Schedule processing and send risk notifications.
Observability	CloudWatch + X-Ray	Monitor, alert, log, and trace the application.

8. Functional Requirements

14. The system must accept transaction CSV files.
15. The system must validate required transaction fields and values.
16. The system must prevent invalid rows from entering the main transaction ledger.
17. The system must store valid transactions.
18. The system must calculate a risk score using configured rules.
19. The system must classify transactions or accounts by risk level.
20. The system must provide an endpoint for retrieving transactions.
21. The system must provide an endpoint for retrieving risk insights.
22. The system must provide an endpoint for scoring a transaction.
23. The system must support scheduled batch risk scoring.
24. The system must generate an alert when the configured risk threshold is breached.
25. The system must record application errors and operational metrics.

9. Non-Functional Requirements

- Security: credentials must never be hardcoded in source code.
- Maintainability: application code must be separated into logical modules.
- Reliability: failed processing must be logged and handled predictably.
- Performance: frequently requested risk scores should use an appropriate fast-access data store.
- Scalability: event-driven components should support variable transaction volumes.

- Observability: important application operations must generate structured logs and metrics.
- Testability: business logic must be designed so it can be unit tested independently.
- Cost awareness: compute and storage choices should avoid unnecessary idle infrastructure.

10. Initial Risk-Scoring Rules

Risk Rule	Condition	Flag
High Amount	Transaction amount > R50 000	HIGH_AMOUNT
High Velocity	More than 10 transactions in one hour	HIGH_VELOCITY
Cross-Border	Transaction meets the configured cross-border condition	CROSS_BORDER

The initial scoring model will be rule-based and explainable. The exact points assigned to each rule and the LOW, MEDIUM, and HIGH classification thresholds will be defined before implementing the scoring module.

11. Initial Data Model

The relational data model will begin with the following core entities:

Entity	Purpose	Example Key
Customer	Stores customer information	customer_id
Account	Stores accounts belonging to customers	account_id
Transaction	Stores individual transaction records	transaction_id

Core relationship: Customer -> Account -> Transaction

Risk-score records will be stored separately for fast lookup, using an access pattern based on account and date where appropriate.

12. Initial API Design

Method	Endpoint	Purpose
GET	/health	Confirm that the application is running.
GET	/transactions	Retrieve transaction records.
GET	/insights	Retrieve risk and transaction insights.
POST	/score	Calculate a risk score for supplied transaction data.

13. Repository Structure

```
fintrust-risk-system/
|-- src/
|   |-- ingest/
|   |-- scoring/
|   |-- api/
|   `-- shared/
|-- infrastructure/
|   |-- iam/
```

```
|   |-- s3/
|   `-- rds/
|-- tests/
|   |-- unit/
|   `-- integration/
|-- docs/
|-- .env.example
|-- .gitignore
|-- requirements.txt
`-- README.md
```

14. Git Branching Strategy

Branch	Purpose	Rule
main	Production-ready code and releases	Merge through reviewed pull requests.
develop	Integration branch	Receives completed feature work.
feature/<name>	One new feature or component	Delete after successful merge.
fix/<name>	Bug correction	Keep short-lived and validate before merge.

15. Environment and Security Standards

- Use a local .env file for local development and add it to .gitignore.
- Commit only .env.example with placeholder configuration values.
- Never commit database passwords, AWS credentials, API keys, or other secrets.
- Use IAM roles for AWS workloads instead of embedding AWS credentials in code.
- Use SSM Parameter Store for protected application configuration in AWS.
- Scope IAM permissions to the resources and actions each component actually requires.
- Use encryption for stored and transmitted sensitive data where appropriate.
- Keep development and production configuration separate.

16. Development Plan

Phase	Focus	Expected Deliverable
1	Foundation	Repository, virtual environment, configuration, project structure.
2	Data Layer	Database schema, SQLAlchemy models, and database connection.
3	Ingestion	CSV parsing, validation, quarantine, and persistence.
4	Risk Engine	Risk rules, scoring, classification, and unit tests.
5	REST API	Health, transactions, insights, and scoring endpoints.
6	AWS Integration	S3, Lambda, DynamoDB, and API Gateway integration.

7	Workflow & Alerts	Step Functions, EventBridge, and SES.
8	Quality Engineering	Unit, integration, API, and end-to-end testing.
9	Observability	CloudWatch logging, metrics, alarms, and X-Ray.
10	Deployment	Cloud deployment, validation, and final documentation.

17. SAA-C03 Connection

SAA-C03 Domain	FinTrust Examples	Learning Connection
Secure Architectures	IAM, encryption, SSM, protected resources	Secure application and service integration.
Resilient Architectures	Retries, error handling, Multi-AZ considerations	Design predictable failure handling.
High-Performing Architectures	DynamoDB, Lambda scaling, RDS Proxy	Match services to access and workload patterns.
Cost-Optimised Architectures	Serverless compute and storage lifecycle	Reduce idle infrastructure and select efficient services.

18. Definition of Done

- Valid transaction data can be ingested and persisted.
- Invalid data is rejected or quarantined correctly.
- Risk rules produce the expected results.
- Risk scores can be stored and retrieved.
- Required REST endpoints work and are documented.
- AWS integrations operate as designed.
- Scheduled processing and alerting work.
- Automated tests pass at the agreed quality threshold.
- Application logs, metrics, and alarms are available.
- Security controls are implemented.
- Deployment steps are documented and repeatable.
- The repository contains complete technical documentation.

19. Pre-Build Checklist

- Confirm project objective, scope, and requirements.
- Confirm the initial risk rules and scoring thresholds.
- Create the Git repository and branching structure.
- Create the Python project structure and virtual environment.
- Create .gitignore and .env.example.
- Design the PostgreSQL schema.
- Prepare sample CSV transaction data.
- Confirm the API contract.
- Confirm the AWS account, region, and development permissions.
- Create the README and architecture documentation.

- Begin Phase 1 only after the foundation decisions are agreed.

20. First Development Milestone

The first milestone is a working local backend foundation. Before integrating AWS services, the team should have a running Python application, a controlled development environment, an initial database model and connection, sample transaction data, a health endpoint, basic tests, and the source code under Git version control.